Mem Address 8 bits (Address of RAM)	Content of Memory (8bit) (Contents of RAM)	Active Signals for the Execute Cycle	PC Contents Current Mem From Which Code is to be Fetched.	Semantics of the Code Executed.
0	movi r0	Ck3: EPC, LMR, IPC SRA ← R0/0	1 at ↑ Ck3 * 2 at ↓ Ck3 □	[R0] ← 10
1	10 *	Ck4: RD, LRG, END*	2 at ↑ Ck4	
2	mov r0	Ck3: ERG, LAR, END SALU ← PASS0	3 at ↑ Ck3*	[AR] ← [R0]=10
3	adi	Ck3: EPC, LMR, IPC SALU ← ADD	4 at ↑ Ck3* 5 at ↓ Ck3 □	[AR] ← [AR]+1 = 10+1 = 11
4	01 *	Ck4: RD, LOR Ck5: EAR, LAR, END	5 at ↑ Ck4. 5 at ↑ Ck5	
5	movd r1	Ck3: EAR, LRG, END SRA ← R1/1	6 at ↑ Ck3*	[R1] ← [AR] [R1] = 11
6	stop	Ck3: END, STOPCLOCK.	0	End of Code

* → END can also be written as RMS, which signals end of execution of that instruction and Reseting the Memory Sequence Register (MSR, will come to it later).

* → At ↑ Ck3 the PC always holds the address of the next instruction instead of the current instruction being executed. This is because during the FETCH cycle the PC got its register value incremented to hold the address of the next memory location.

□ → At ↓ Ck3. The PC is pointing to the address that holds DATA and NOT CODE. Hence it gets INCREMENTED

* Memory Locations 1 and 4 hold DATA and not CODE. Since in this processor there IS no separate Program and Data Memory unlike the classical VON Neumann or Harvard Processor Architectures

* movi R0, 10 Clk3: EPC, LMR, IPC, SDA ← RD
Clk4: RD, LDA, END/RMS, SRC ← RD

At the beginning of Clk3 PC holds address 1 which has DATA and not CODE. Also this contents of 1, needs to be moved to the DATA BUS and subsequently to R0.

Clk3: EPC, LMR, IPC, SDA ← D: Loads the contents of [PC] to DB. or 1 is loaded onto the DATA BUS on the ↑ of Clk3.

On ↓ of Clk3 the Memory Address Register or simply Memory Register loads 1 from the Data Bus into its contents. Like magic before the arrival of the next ↑ (Clk4) it accesses the contents of Memory Location 1 and keeps it Ready to load it to the Data Bus.

Also at ↓ of Clk3: [PC] ← [PC]+1 = 2 and holds the address of the Next Instruction

Ck4: RD, LRA, END; SRA ← RD

At Ck4 of Execute cycle the contents of the external memory I is Read by the RD signal and loaded to the DATA BUS. Or simply IO is loaded to the Data Bus

At ↓ Ck4, LRA, SRA ← RD ensures that IO is transferred or Loaded to RD from the DATA BUS.

Similar analysis applies for add 01.

The FETCH and DECODE Cycle:

Ck1 and Ck2 are always reserved for fetching the code from the External Memory / RAM.

Ck1: EPC; LMR, IPC.

Ck2: RD, LIR, LMS

Ck1: Loads the contents of the PC Register (address of the current memory location, whose contents needs to be fetched and executed) to the DB at ↑ Ck1

At ↓ Ck1: The MR is loaded with [PC] and the corresponding memory location is accessed and its content kept ready to be loaded to the DB at next clock.
(The ckt corresponding to this is something that can be designed by you folks). Also at ↓ Ck1 the [PC] is incremented by 1 via IPC so that it can point to or hold the address of the next memory location

Ck2: RD, LIR, LMS. The contents of the memory location that holds the code is loaded to the DB.

Essentially the CODE is loaded to the DB at ↑ of Ck2.

At ↓ of Ck2: The code is loaded to IR and MS register through LIR, LMS.

The IR Register decodes the Instruction of the CODE and MS Register sequences the signals that need to go high for the execute cycle of the Instruction.

The execute cycle can take minimum of 1 clock and maximum of 5 clocks as in case of cd <FL> instruction (Next class).